ChatGPT

HiDiffusion 两周执行计划（本科生可交付版）

执行摘要

本计划将原“HiDiffusion 改造方案”压缩为 **两周内可完成、可复现、可量化、可写简历** 的工程项目：只做两件最值钱的事——**benchmark runner（速度/吞吐/峰值显存/失败率）** 与 **auto_config（按显存预算+目标分辨率给出推理策略）**。
对齐机器学习/生成模型岗位（熟悉 diffusion + 工程化改造 + 可复现）要求：交付物以 **可运行命令、CSV/Markdown 报告、示例图片、策略 JSON、README 一键复现** 为核心，确保面试能讲清楚“公平对比”“性能测量”“显存策略”“工程化可复现”。
计划默认你具备基本 Python/PyTorch 与 Git 能力；不额外假设算力，但提供 **CPU/4GB GPU 的 smoke test** 与 **Colab/云 GPU 替代步骤**。
背景依据以官方项目与论文为主：HiDiffusion 的一行集成与建议配置见 README（来源：`https://github.com/megvii-research/HiDiffusion/blob/main/README.md`），扩散模型与高分辨率/效率问题背景可参考 HiDiffusion 论文（来源：`https://arxiv.org/abs/2311.17528`）以及 diffusers 生态（来源：`https://github.com/huggingface/diffusers`）。

范围与成功标准

项目范围压缩原则

两周版本只保留“对岗位最有用、且本科生风险最低”的部分：

- **保留**：可复现 benchmark、性能/显存测量、失败案例统计、规则型 auto strategy、文档与最小测试。
- **不做**：FID/CLIPScore 等重评测、复杂跨版本兼容层、4096×4096 极限挑战、多任务（ControlNet/Inpainting）全面覆盖。

理由：HiDiffusion 价值点在“高分辨率与效率”，最容易拿到硬指标，并且其设计是“tuning-free + 与 diffusers 管线集成”（来源：`https://github.com/megvii-research/HiDiffusion/blob/main/README.md`；论文：`https://arxiv.org/abs/2311.17528`）。两周内用 **速度/显存/失败率** 就能写出高质量简历条目。

成功标准（两周结束时你必须能做到）

你能在 README 提供 **一键复现命令**，并生成下面这些“可量化产物”：

- 运行 `scripts/benchmark/run_benchmark.py` 后：
- `results/*.csv` 至少包含列：`latency_sec`, `throughput_img_s`, `peak_vram_mb`（以及必要的配置列）
- `results/report.md` 自动生成对比表（baseline vs enable_hidiffusion）与结论段落
- `results/samples/` 输出若干张示例图片（或 grid），用于 README 展示
- 运行 `scripts/benchmark/auto_config.py` 后：
- 输出策略 `strategy.json`，包含：`enable_hidiffusion`, `enable_xformers`, `enable_cpu_offload`, `enable_vae_tiling`, `steps`
- 通过 smoke test：
- CPU/低算力模式能跑通“生成 CSV+MD+策略 JSON”（即使不真实跑大模型）
- 推荐 GPU 模式能跑通至少 3 个分辨率点对比并生成最终报告

交付物清单与实现要点

下面按“必须交付物”逐一给出：**关键实现要点、关键文件/函数、测试方法、验收标准**。路径严格按你的要求。

交付物一：可复现 benchmark runner

路径：`scripts/benchmark/run_benchmark.py`
目标：对比 baseline diffusers vs +HiDiffusion 的推理性能，并输出 CSV + Markdown 报告。HiDiffusion 典型用法是构建 diffusers pipeline 后调用 `apply_hidiffusion(pipe)`（来源：`https://github.com/megvii-research/HiDiffusion/blob/main/README.md`），因此 bench 的核心是“开关 patch + 固定其他变量”。

必须支持的 CLI 参数

- `--model_id`：如 `stabilityai/stable-diffusion-xl-base-1.0`（示例来源：`https://github.com/megvii-research/HiDiffusion/blob/main/README.md`）
- `--enable_hidiffusion`：bool（或 `--enable_hidiffusion 0/1`）
- `--height`, `--width`
- `--steps`
- `--scheduler`：至少支持 `ddim`（示例来源：`DDIMScheduler` 用法见 HiDiffusion README：`https://github.com/megvii-research/HiDiffusion/blob/main/README.md`）
- `--seeds`：例如 `0,1,2`
- 建议补充：`--prompt_file`（JSON）、`--out_dir`、`--device cuda/cpu`、`--num_warmup`、`--dtype fp16/fp32`

CSV 与报告输出（最低要求）

- CSV 至少包含：
- 配置列：`model_id,scheduler,height,width,steps,seed,enable_hidiffusion`
- 指标列：`latency_sec,throughput_img_s,peak_vram_mb`
- 建议附加：`success,fail_reason,timestamp,torch_version,diffusers_version,hidiffusion_version`
- Markdown 报告 `results/report.md` 至少包含：
- 汇总表：按分辨率聚合的平均 latency / throughput / peak_vram（baseline vs hidiffusion）
- 失败统计：OOM/下载失败/其他异常次数
- 结论段：例如“从 1536 起，HiDiffusion 在相同步数下 latency 平均降低 X%...”

关键实现要点（一定要做对）

1) **公平对比**：固定 prompt/seed/steps/scheduler/guidance_scale，只切换 `enable_hidiffusion`。
2) **GPU 计时正确**：CUDA 是异步的，必须 `torch.cuda.synchronize()` 后再计时、结束再 synchronize。
3) **峰值显存**：每次 run 前 `torch.cuda.reset_peak_memory_stats()`，结束后用 `torch.cuda.max_memory_allocated()`。
4) **warmup**：至少 1 次 warmup（JIT/缓存）不计入结果，避免首轮偏慢。
5) **失败不中断**：捕获 OOM/网络/模型加载异常，写入 `fail_reason`，继续跑下一组配置。

建议的关键函数拆分（便于两周内写干净）

- `parse_args()`
- `load_prompts(prompt_file) -> list[dict]`
- `build_scheduler(name, model_id)`

- `build_pipeline(model_id, scheduler, dtype, device)`
- `apply_hidiffusion_if_needed(pipe, enable_hidiffusion)`
- `run_one(prompt, seed, height, width, steps) -> (image, metrics)`
- `write_csv(rows, path)`
- `render_markdown_report(rows, path)`

核心计时/显存记录逻辑（关键代码片段）

```
import time, math
import torch

def measure_generate(pipe, prompt, negative_prompt, seed, height, width, steps):
    device = pipe._execution_device if hasattr(pipe, "_execution_device") else "cuda"
    gen = torch.Generator(device=device).manual_seed(seed)

    # 显存与计时准备
    peak_vram_mb = math.nan
    if torch.cuda.is_available():
        torch.cuda.reset_peak_memory_stats()
        torch.cuda.synchronize()
    t0 = time.perf_counter()

    try:
        out = pipe(
            prompt,
            negative_prompt=negative_prompt,
            height=height,
            width=width,
            num_inference_steps=steps,
            generator=gen,
        )
        if torch.cuda.is_available():
            torch.cuda.synchronize()
        t1 = time.perf_counter()

        latency = t1 - t0
        throughput = 1.0 / latency if latency > 0 else math.nan

        if torch.cuda.is_available():
            peak_vram_mb = torch.cuda.max_memory_allocated() / (1024 ** 2)

        return out.images[0], {
            "success": True,
            "latency_sec": latency,
            "throughput_img_s": throughput,
            "peak_vram_mb": peak_vram_mb,
            "fail_reason": "",
        }

    except torch.cuda.OutOfMemoryError:
        if torch.cuda.is_available():
```

```
            torch.cuda.empty_cache()
        return None, {
            "success": False,
            "latency_sec": math.nan,
            "throughput_img_s": math.nan,
            "peak_vram_mb": peak_vram_mb,
            "fail_reason": "oom",
        }
    except Exception as e:
        return None, {
            "success": False,
            "latency_sec": math.nan,
            "throughput_img_s": math.nan,
            "peak_vram_mb": peak_vram_mb,
            "fail_reason": f"exception:{type(e).__name__}",
        }
```

测试方法与验收

- **CPU smoke test（不跑大模型）**：提供 `--dry_run` 或 `--mock_pipe` 模式：跳过真实模型加载，返回假图/假指标，但完整产出 CSV+MD。
- **4GB GPU smoke test（真实跑最小）**：选小分辨率（512×512）、步数 1–4、只跑 1 个 prompt、1 个 seed；可用 CPU offload 降显存。
- **最终验收（推荐 GPU）**：同一张卡上跑 3 个分辨率点（如 1024/1536/2048）× 2–3 seeds × baseline/hidiffusion，生成最终 `report.md`。

交付物二：auto_config.py（基于显存预算与目标分辨率的策略推荐）

路径：`scripts/benchmark/auto_config.py`
目标：输入 `vram_budget_gb` 与 `target_resolution`（或 height/width），输出推荐策略 JSON：`enable_hidiffusion/xformers/cpu_offload/vae_tiling/steps`

HiDiffusion README 明确提到可结合 xformers、CPU offload、VAE tiling 等降低显存并提升效率（来源：`https://github.com/megvii-research/HiDiffusion/blob/main/README.md`），因此 auto_config 以“把这些开关工程化”为主。

必须支持的 CLI 参数

- `--vram_budget_gb`（float）
- `--height`, `--width`（或 `--target_resolution 2048`）
- 建议补充：`--prefer_speed`（默认 True）、`--model_family sdxl/sd15`（不同模型开关阈值可不同）
- 输出：`--out strategy.json`（默认 stdout 也可）

策略 JSON 最低字段

```
{
 "enable_hidiffusion": true,
 "enable_xformers": true,
```

```
  "enable_cpu_offload": false,
  "enable_vae_tiling": true,
  "steps": 20
}
```

规则型策略选择（两周版“够用即可”）

两周版不做复杂学习/探测，采用“可解释的阈值规则”，并在 README 说明规则依据来自你第一周 benchmark 的观察（为什么某分辨率开始收益显著、为什么某预算必须 offload）。

auto_config 伪代码（关键代码片段）

```
def recommend_strategy(vram_budget_gb: float, h: int, w: int) -> dict:
    px = h * w
    mp = px / 1e6   # mega-pixels

    # 1) 基础开关
    s = {
        "enable_hidiffusion": False,
        "enable_xformers": True,            # 优先尝试；失败则在运行侧 fallback
        "enable_cpu_offload": False,
        "enable_vae_tiling": False,
        "steps": 20,
    }

    # 2) 分辨率越高越倾向开启 HiDiffusion 与 VAE tiling
    if mp >= 2.0:   # ~ 1400x1400 以上
        s["enable_hidiffusion"] = True
    if mp >= 3.0:   # ~ 1732x1732 以上
        s["enable_vae_tiling"] = True
        s["steps"] = 12   # 少步数保证可跑；也可按你的 bench 调整

    # 3) 显存预算不足时启用 offload，并进一步降步数/降分辨率建议
    if vram_budget_gb <= 8:
        s["enable_cpu_offload"] = True
        s["steps"] = min(s["steps"], 8)

    # 4) 预算很高则以质量/稳定为主
    if vram_budget_gb >= 24:
        s["enable_cpu_offload"] = False
        s["steps"] = max(s["steps"], 20)

    return s
```

注意：阈值（比如 mp 多少启用 tiling、vram_budget_gb 的分档）需要你用 benchmark 结果做一次校准。两周版追求“可解释+可复现”，不追求最优。

测试方法与验收

- 单元测试：输入不同 (vram_budget_gb, height, width) 组合，断言输出 JSON 字段齐全且类型正确。
- 运行验收：`python scripts/benchmark/auto_config.py --vram_budget_gb 12 --height 2048 --width 2048 --out results/strategy.json` 能生成文件；README 给出示例输出。

交付物三：README.md（最小示例 + 一键复现命令）

路径：仓库根目录 `README.md`（你 fork 的 HiDiffusion 分支上修改/增加）
目标：任何人按 README 可以在 10–20 分钟内跑通最小示例，并复现你给出的 benchmark 报告生成流程。

README 必须包含的内容（建议按这个顺序写）

1) **项目一句话简介**：你做了什么（benchmark + auto strategy）
2) **环境与依赖**：pip install 列表（HiDiffusion README 的依赖基线：torch、diffusers、transformers、accelerate、xformers；来源：`https://github.com/megvii-research/HiDiffusion/blob/main/README.md`）
3) **最小示例（生成 1 张图）**：展示 baseline 与 enable_hidiffusion 的对比开关（HiDiffusion 的 `apply_hidiffusion(pipe)` 用法来源同上）
4) **一键 benchmark 命令**：生成 CSV + MD + samples
5) **一键 auto_config 命令**：生成策略 JSON
6) **结果展示**：贴 report.md 的关键表格截图或 figures（推荐放 2–3 张）
7) **可复现声明**：说明固定 seed、记录版本号、warmup 规则

一键命令示例（README 中可直接复制）

```
# 1) 安装
pip install -r requirements.txt

# 2) 跑 benchmark（示例：3 个分辨率，3 个 seed）
python scripts/benchmark/run_benchmark.py \
 --model_id "stabilityai/stable-diffusion-xl-base-1.0" \
 --scheduler ddim \
 --height 1024 --width 1024 \
 --steps 20 \
 --seeds "0,1,2" \
 --enable_hidiffusion 0 \
 --out_dir results/baseline_1024

python scripts/benchmark/run_benchmark.py \
 --model_id "stabilityai/stable-diffusion-xl-base-1.0" \
 --scheduler ddim \
 --height 1024 --width 1024 \
 --steps 20 \
 --seeds "0,1,2" \
 --enable_hidiffusion 1 \
 --out_dir results/hidiffusion_1024

# 3) 自动生成总报告（如果你把聚合写在 runner 里，可省略此步）
# python scripts/benchmark/merge_and_report.py ...
```

```
# 4) 生成策略 JSON
python scripts/benchmark/auto_config.py \
  --vram_budget_gb 12 \
  --height 2048 --width 2048 \
  --out results/strategy_12gb_2048.json
```

交付物四：简历条目模板与面试问答要点

路径建议： `docs/resume_and_interview.md`（也可直接写在 README 末尾）
目标： 你投简历时可以直接复制粘贴；面试时能 2 分钟讲清楚技术点与工程点。

简历条目模板（中文，可直接替换数字）

- **HiDiffusion 推理评测与自适应配置工具（PyTorch/diffusers）**：实现可复现 benchmark runner，对比 baseline 与 HiDiffusion 在多分辨率/多 seed 下的 **latency、throughput、peak VRAM 与失败率**，自动生成 **CSV/Markdown 报告**与可视化结果；在 `[你的 GPU 型号/显存]` 上实现 **推理延迟降低 X% / 峰值显存降低 Y% / 可稳定支持到 Z×Z 分辨率**（按你的实测填写）。
- 工程化改造：提供 `auto_config.py`，按 **显存预算与目标分辨率** 推荐 `HiDiffusion/xformers/CPU offload/VAE tiling/steps` 策略，并落盘元数据（seed、版本、参数），提升可复现性与排障效率。

面试问答要点（三个高频，含答案要点）

1) **你怎么保证 benchmark 公平?**
- 固定 prompt/seed/steps/scheduler/guidance；只切换 `enable_hidiffusion`。
- GPU 计时用 `torch.cuda.synchronize()` 避免异步误差；设 warmup 排除首轮抖动。
- 输出中记录 torch/diffusers/hidiffusion 版本与硬件信息，保证可复现。

2) **为什么主要指标选 latency、throughput、peak VRAM?**
- 对“产品级 AIGC 推理”最直接：线上关心响应时间、吞吐、显存成本与失败率。
- HiDiffusion 的核心价值点就是提升高分辨率推理效率（论文动机：`https://arxiv.org/abs/2311.17528`），因此这些指标最对齐。

3) **auto_config 的规则依据是什么？会不会拍脑袋?**
- 两周版采用可解释阈值规则，阈值来自第一周 benchmark 观察（例如某分辨率开始明显 OOM/明显变慢）。
- 明确声明局限：不同 GPU/驱动/模型 family 阈值不同；策略模块设计为可扩展（未来可接入探测或学习型策略）。

环境、依赖与低算力替代路径

最小可行环境（MVP）

- Python：3.10（或 3.8+，以 HiDiffusion 依赖说明为参考；来源：`https://github.com/megvii-research/HiDiffusion/blob/main/README.md`）
- PyTorch：≥ 1.13.1（来源同上；实际建议用更近版本以匹配较新 CUDA）
- diffusers：≥ 0.25.0（来源同上）
- 必要 pip 包（写进 `requirements.txt`）：
- `diffusers>=0.25.0`
- `transformers`

- `accelerate`
- `hidiffusion`（或你 fork 仓库内本地开发）
- `safetensors`
- `numpy`，`pandas`（汇总 CSV/表格）
- `matplotlib`（画图，可选）
- `tqdm`（进度条，可选）

  说明：HiDiffusion 一行集成 diffusers pipeline 的用法与依赖建议见其 README（来源：`https://github.com/megvii-research/HiDiffusion/blob/main/README.md`）。

推荐硬件（用于最终验证）

- 推荐：NVIDIA GPU ≥ **12GB 显存**（能更稳跑 1024+ 分辨率）；如要做 2048 多点 benchmark 更建议 16–24GB。
- 备选：8GB 也可尝试，但需更激进策略（CPU offload、降低 steps、降低分辨率），并接受更慢。

低算力替代方案

- **CPU 模式（仅 smoke test）**：使用 `--dry_run/--mock_pipe`，验证脚本产物链路（CSV/MD/JSON）完整，不追求真实扩散推理。
- **4GB GPU（真实 smoke test）**：
- 使用更低分辨率（如 512×512）、更少步数（1–4）、更少样本（1 prompt × 1 seed）
- 启用 `enable_cpu_offload` 与 `enable_vae_tiling`（策略由 `auto_config.py` 输出）
- **Colab/云 GPU 示例命令**（Notebook 单元可直接跑）

```
!pip -q install diffusers>=0.25.0 transformers accelerate safetensors hidiffusion
# 可选：如果环境支持
# !pip -q install xformers

!python scripts/benchmark/run_benchmark.py \
 --model_id "stabilityai/stable-diffusion-xl-base-1.0" \
 --scheduler ddim \
 --height 1024 --width 1024 \
 --steps 10 \
 --seeds "0" \
 --enable_hidiffusion 1 \
 --out_dir results/colab_smoke
```

  提醒：部分模型权重在 Hugging Face 可能需要登录/同意协议才能下载；若下载受限，就把 `--model_id` 换成你可访问的开源模型，并在 README 记录你实际测试的模型与版本（diffusers 项目主页：`https://github.com/huggingface/diffusers`）。

日程安排

下面给出 **共 14 天、总计约 77 小时** 的执行表（按“全职等价 ≤ 80 小时”控制）。如果你是兼职（例如每天 2–3 小时），可把“每日目标”按顺序拉长到 3–4 周，但交付物不变。

```
flowchart TD
  A[Day 1-2: 环境与最小示例跑通] --> B[Day 3-7: Benchmark Runner + 报告/可视化]
  B --> C[Day 8-10: Auto Config 规则 + 输出策略 JSON]
```

```
C --> D[Day 11-12: Smoke Test + README 一键复现]
D --> E[Day 13-14: 最终报告 + 简历条目 + 面试演练]
```

两周详细日程表（按天）

天	预计小时	当日目标	里程碑产物	当日验收标准（最小可运行产出）
Day 1	6h	fork HiDiffusion，建分支；装环境；跑通官方示例（baseline 与开启 HiDiffusion）	本地能生成两张图	`apply_hidiffusion(pipe)` 开/关均能生成图片并保存
Day 2	6h	固定实验协议：模型/分辨率/steps/scheduler/seed/promptset；写 prompt JSON	`scripts/benchmark/promptset_v1.json`	10 条 prompt + 1 个 negative_prompt；参数表写入 notes/README 草稿
Day 3	6h	写 `run_benchmark.py` v0：单条 prompt 单个 seed 产出一行 CSV	初版 runner	命令行跑一次能生成 `raw.csv`，包含 latency/throughput/peak_vram 列
Day 4	6h	扩展 runner：支持 prompts × seeds；异常捕获；warmup；输出 samples 图片	runner 可批跑	小矩阵（2 prompts × 2 seeds × baseline/hidiffusion）能跑完且不中断
Day 5	5h	跑第一版 benchmark（建议 5 prompts × 2 seeds × 1–2 分辨率）；写 report.md v0	`results/report.md` v0	report.md 有汇总表与初步结论段
Day 6	5h	加聚合逻辑与可视化脚本（可选）；输出 figures	`results/figures/*.png`	至少 2 张图：分辨率-延迟、分辨率-显存（哪怕很粗）
Day 7	5h	代码清理：拆函数；记录版本信息；补充 fail_reason；完善 CSV schema	runner v1	runner 能输出包含版本/硬件信息的 CSV；结构稳定
Day 8	6h	设计 auto_config 规则分档；定义策略 JSON schema	规则表	给 3 组输入（8GB/12GB/24GB × 1024/2048）能手算出策略
Day 9	6h	实现 `auto_config.py` + CLI；落盘 JSON；（可选）在 run_benchmark 中读取策略	auto_config 可运行	`auto_config.py` 能输出字段齐全的 `strategy.json`

天	预计小时	当日目标	里程碑产物	当日验收标准（最小可运行产出）
Day 10	5h	用 benchmark 结果校准阈值；在 README 解释依据（“从某分辨率开始收益显著”）	有依据的策略	README 草稿写清楚策略规则与依据；至少引用你自己的 CSV 统计结果
Day 11	5h	写 smoke tests：CPU dry_run +（可选）GPU optional；保证产物链路	`tests/test_smoke.py`	`pytest` 在 CPU 上能跑通（不下载大模型也能生成 CSV+MD+JSON）
Day 12	5h	写正式 README：安装、最小示例、benchmark、auto_config、一键命令、结果图	`README.md` 完整	新机器按 README 能跑 smoke benchmark 并生成 report
Day 13	5h	跑最终 benchmark（3 分辨率 × 2–3 seeds × baseline/hidiffusion）；整理最终 report	最终报告	`results/report.md` 有明确量化提升（X%）或至少完整对比表
Day 14	5h	写简历条目与面试问答；做 2 分钟项目陈述；最后清理 repo	`docs/resume_and_interview.md`	你能用“问题-方法-指标-结果-局限”讲清楚且 README 可发链接

风险与验证策略

主要风险点与缓解措施（至少五项）

1) **模型权重下载失败/需要鉴权**
- 缓解：提前在 Day 1 完成模型下载验证；准备至少 1 个可访问的备选 `model_id`；在 README 写清你使用的模型与下载前置条件。

2) **OOM（尤其高分辨率/SDXL）**
- 缓解：把最终验收分辨率设为“可跑的三点”（例如 1024/1536/2048）而非硬冲更高；auto_config 默认启用 `vae_tiling`/`cpu_offload` 的保守策略；失败要记录在 CSV，不要硬扛。

3) **xformers 安装失败或平台不支持**
- 缓解：xformers 设为“可选优化”；策略输出里可以建议启用，但运行时要有 fallback：捕获导入/启用失败则继续跑并在 metadata 记录“xformers_unavailable”。

4) **diffusers API 变动导致代码不兼容**
- 缓解：requirements 固定最小版本（例如 `diffusers>=0.25.0`，与 HiDiffusion README 一致）；在 CSV/metadata 记录 diffusers 版本；在 README 建议用同版本复现。

5) **性能测量不准（CUDA 异步、warmup、缓存影响）**
- 缓解：强制 synchronize；设置 warmup 不计入；多 seed 平均；报告中注明“冷启动/热启动差异”和“统计口径”。

6) **时间不足（两周很紧）**
- 缓解：优先级严格：先保证 runner 输出 CSV+MD（Day 3-5）；auto_config 先规则表再代码（Day 8-9）；图表与高级功能都设为可选。

7) **结果不明显（HiDiffusion 提升在你配置下不显著）**
- 缓解：报告不仅写“提升”，也写“观察到的 trade-off 与适用条件”；把价值转成“工程化评测 + 策略化开关 + 可复现工具链”，这对岗位同样加分。

测试与最终验证方案

- **CPU/低算力 smoke test**（目标：验证“工程链路可复现”）
- `python scripts/benchmark/run_benchmark.py --dry_run 1 ...`
- 验收：生成 `raw.csv` + `report.md` + `strategy.json`，CSV 字段齐全，report 有汇总表。
- **4GB GPU smoke test**（目标：验证“真实推理能跑通”）
- 使用低分辨率 + 少步数 + 少样本：512×512、steps=2、1 prompt、1 seed
- auto_config 输出应倾向 `cpu_offload=True`、`steps<=8`。
- **推荐 GPU 最终验证**（目标：产出“可写简历的量化数字”）
- 至少：3 个分辨率点 × 2–3 seeds × baseline/hidiffusion
- 产出：最终 `report.md`（含提升百分比）+ figures + samples

以上执行计划严格覆盖你的“必须交付物”四项要求，并把原方案压缩为两周内最稳、最能写简历的工程闭环：**可复现 benchmark（CSV+MD）+ 可解释 auto strategy（JSON）+ README 一键复现 + 简历与面试材料**。背景依据优先来自官方 HiDiffusion README（`https://github.com/megvii-research/HiDiffusion/blob/main/README.md`）、diffusers 生态（`https://github.com/huggingface/diffusers`）与论文（`https://arxiv.org/abs/2311.17528`）。